```
 __       __            ______    ______  ________
|  \     /  \          /      \  /      \|        \
| $$\   /  $$  _______ |  $$$$$$\|  $$$$$$\$$$$$$$$
 | $$$\ /  $$$ /       \| $$   \$$| $$   \$$  | $$
 | $$$$\  $$$$|  $$$$$$$| $$      | $$        | $$
 | $$\$$ $$ $$| $$      | $$   __ | $$   __   | $$
 | $$ \$$$| $$| $$_____ | $$__/  \| $$__/  \  | $$
 | $$  \$ | $$ \$$     \ \$$    $$ \$$    $$  | $$
  \$$      \$$  \$$$$$$$  \$$$$$$   \$$$$$$    \$$
```

Controlled Concurrency Testing for Scala

Philipp Haller and Oskar Grönman

Digital Futures and EECS

KTH Royal Institute of Technology, Stockholm, Sweden

Scalar 2026, March 26-27, 2026, Warsaw, Poland

Motivation

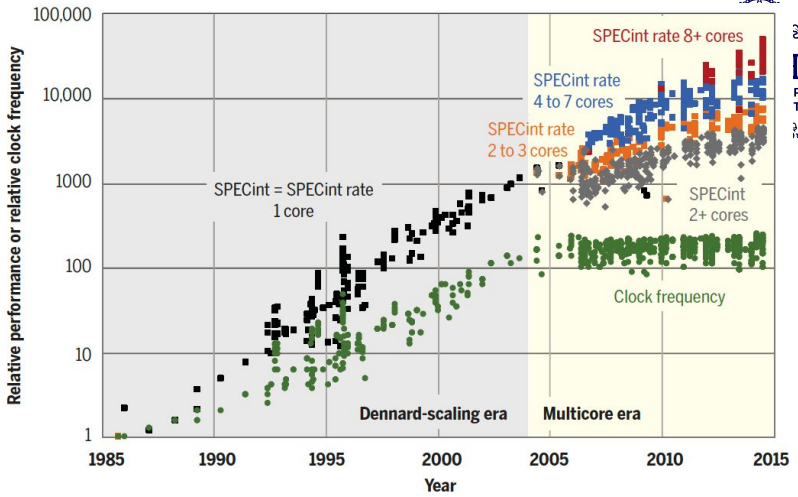


Fig. 2. SPECint (largely serial) performance, SPECint-rate (parallel) performance, and clock-frequency scaling for microprocessors from 1985 to 2015, normalized to the Intel 80386 DX microprocessor in 1985. Microprocessors and their clock frequencies were obtained from the Stanford CPU database (*56*).

- The rise of multi-core processors
  - Clock frequencies doubled roughly every 18-24 months
  - From ~1 MHz (1980) to 3-4 GHz (2003-2004)
  - Dennard scaling: supply voltage scales down with transistor size, keeping power density constant
  - ***Power wall:*** voltage couldn't scale further and leakage current became dominant ⇒ **end of clock frequency scaling** ⇒ higher performance requires more processor cores
- Cloud computing requires concurrency
  - Concurrency required to hide latency of I/O
  - Aggregate workloads are inherently parallel
  - AI needs horizontal scaling beyond multi-cores

Concurrency unavoidable
at every level of the stack!

Concurrent programming is challenging

- Coordination of concurrent tasks requires either **shared memory** or **message passing**
- Nondeterminism makes this difficult
  - Order of accesses to shared memory
  - Order of message sends/arrivals
- Easy to create bugs
  - Shared memory: "too little locking" ⇒ data races, "too much locking" ⇒ deadlocks
- Very difficult to reproduce bugs
  - Nondeterminism ⇒ indefinite number of runs where bug cannot be observed
  - "Heisenbugs"

Controlled concurrency testing (CCT)

- Helps **find and reproduce concurrency bugs**
- *Bug finding:*
  - Instrumentation of concurrent code, adding calls to a scheduler
  - Scheduler controls order of task execution
  - Systematic exploration of possible task interleavings
  - Record schedules leading to **runtime errors** (exceptions, assertion errors, ...)
- *Bug reproduction:* **deterministic replay** of buggy executions
- CCT a critical tool to ensure reliability of widely-used cloud services
  - Example: large-scale usage of CCT at Microsoft to test Azure cloud services

Pantazis Deligiannis, Aditya Senthilnathan, Fahad Nayyar, Chris Lovett, Akash Lal:
Industrial-Strength Controlled Concurrency Testing for C# Programs with Coyote. TACAS 2023

Example

```scala
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: Data race in practice

- Two futures:
  - Both increment `x`
- Sequential semantics:
  - Output = 2
- What is the result in practice?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: Data race in practice

- Two futures:
  - Both increment `x`
- Sequential semantics:
  - Output = 2
- What is the result in practice?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: Data race in practice

- Two futures:
  - Both increment `x`
- Sequential semantics:
  - Output = 2
- What is the result in practice?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: Data race in practice

- Two futures:
  - Both increment `x`
- Sequential semantics:
  - Output = 2
- Results in practice:
  - Output is sometimes 1
  - Reason: data race

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: What is happening?

- Reassignment not atomic
- Each future executes multiple operations
  - Read
  - Write

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      x = x + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: What is happening?

- Reassignment not atomic
- Each future executes multiple operations
  - Read
  - Write

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      …
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: What is happening?

- Reassignment not atomic
- Each future executes multiple operations
  - Read
  - Write

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      x = tmp + 1
    }
    Future {
      x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: What we (sequentially) expect

```
                 ──────────────────────────────────────────────────────> time

                      |          |           |              |
                      |          |           |              |
                      |          |           |              |
                                             |              |
Future 1:    read(x = 0)  write(x = 1)       |              |
                                             |              |
                                             |              |
                                             |              |

Future 2:                                read(x = 1)  write(x = 2)
```

Example: What can happen

```
                 ──────────────────────────────────────────────────────> time

                      |           |         |           |
                      |           |         |           |
                      |           |         |           |
                                  |                     |
Future 1:    read(x = 0)          |   write(x = 1)      |
                                  |                     |
                                  |                     |
                                  |                     |

Future 2:                 read(x = 0)           write(x = 1)
```

How to trigger the bug more often?

- Bug happens because **both** Futures *read* before *write*
- Introduce delay
  - Future 2 read before Future 1 write

```
                 ──────────────────────────────────────────────────────────> time

                      |                 |           |           |
                      |                 |           |           |
                      |                 |           |           |
                                        |           |
Future 1:    read(x = 0)   sleep()      |           |    write(x = 1)
                                        |           |
                                        |           |
                                        |           |

Future 2:                        read(x = 0)   write(x = 1)
```

Example: w/ sleeps

- Bug happens more often
- But...
  - Still not guaranteed
  - Not reproducible

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      Thread.sleep(2)
      x = tmp + 1
    }
    Future {
     x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: w/ sleeps

- Bug happens more often
- But...
  - Still not guaranteed
  - Not reproducible

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      Thread.sleep(2)
      x = tmp + 1
    }
    Future {
     x = x + 1
    }
  println(s"Counter: $x")
  x
```

Example: w/ sleeps

- Bug happens more often
- But...
  - Still not guaranteed
  - Not reproducible

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      Thread.sleep(2)
      x = tmp + 1
    }
    Future {
     x = x + 1
    }
  println(s"Counter: $x")
  x
```

Overview of McCCT

What is McCCT?

- McCCT: an implementation of CCT for Scala
- Enables a controlled execution of concurrent code
- Core: CCT scheduler
  - Wrapped concurrency libraries call into scheduler
  - Scheduler controls concurrent execution
  - Scheduler can explore different interleavings
  - No two tasks running in parallel to ensure determinism

McCCT: schedulePoints

- Placed before each concurrent operation
- Ask scheduler:
  - Continue?
  - Switch future?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
  x
```

McCCT: schedulePoints

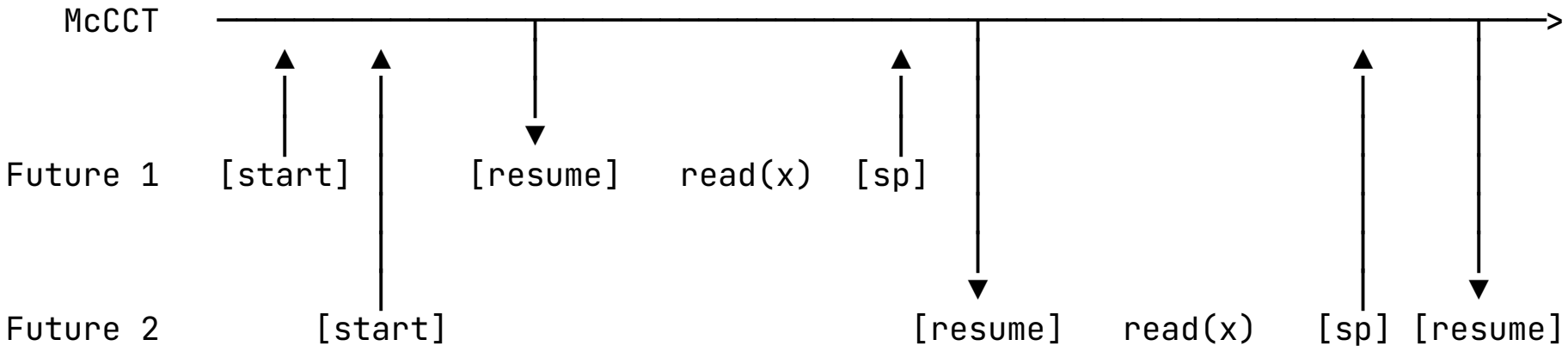
- Placed before each concurrent operation
- Ask scheduler:
  - Continue?
  - Switch future?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
  x
```

McCCT: schedulePoints

- Placed before each concurrent operation
- Ask scheduler:
  - Continue?
  - Switch future?

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
  x
```


McCCT
Future 1
[start]
[resume]
read(x)
[sp]
Future 2
[start]
[resume]
read(x)
[sp]
[resume]

CCT: From chance => search

- Before:
  - Random testing, not reproducible
- After:
  - Search problem
    - Search space: task interleavings
    - Goal: finding errors

McCCT: Reproducibility

- Replay schedules
  - McCCT saves faulty schedules
  - Replaying a schedule yields the same execution ⇒ same bug is observed
  - Deterministic reproduction of bugs

McCCT: Why this matters

- Finding bugs is hard
- Fixing bugs is hard
  - Being able to reproduce them allows us to find a fix
- Did we really fix it?

McCCT: Is the bug fixed?

- Create a fix
  - Re-run the program a number of times
    - We do not encounter the bug
    - Illusion of correctness
  - Not always feasible to check all possible interleavings
- Replay failing schedule
  - With fix applied
  - Bug gone => Fix works

McCCT: Regression testing in practice

```
def counter(): Int =
  @volatile var x = 0
  Async.blocking:
    Future {
      val tmp = x
      schedulePoint()
      x = tmp + 1
    }
    Future {
      ...
    }
  x
```

McCCT:

- Solved three main problems with bug finding concurrent programs
  - Reliably search for bugs
  - Reproduce bugs once found
  - Easily verify fixes

McCCT:

- Solved three main problems with bug finding concurrent programs
  - Reliably search for bugs
  - Reproduce bugs once found
  - Easily verify fixes

McCCT:

- Solved three main problems with bug finding concurrent programs
  - Reliably search for bugs
  - Reproduce bugs once found
  - Easily verify fixes

McCCT:

- Solved three main problems with bug finding concurrent programs
  - Reliably search for bugs
  - Reproduce bugs once found
  - Easily verify fixes

How to search for bugs

- How should we explore schedules?
- Goal
- Example

How to search for bugs

- How should we explore schedules?
- Goal:
  - No duplicate schedules
  - Executes few schedules
  - Still able to find many bugs
- Example

How to search for bugs

- How should we explore schedules?
- Goal:
  - No duplicate schedules
  - Executes few schedules
  - Still able to find many bugs
- Example:
  - Delay Bounding

Exploration algorithms

- Example:
  - Delay Bounding
    - Bounds number of schedules
    - Explores “small deviations” from Round Robin
- McCCT provides API to create algorithms

What if we miss code?

- Did we explore everything?
  - Strategy may ignore paths
  - Hard to translate interleavings into tested statements
- `markers()`
  - Track execution points
  - Make sure we hit certain parts of the code

What if we miss code?

- Did we explore everything?
  - Strategy may ignore paths
  - Hard to translate interleavings into tested statements
- `markers()`
  - Track execution points
  - Make sure we hit certain parts of the code

Other forms of errors

- Data races not only concurrency bug
- Deadlocks
  - Circular waiting
  - Example

Other forms of errors

- Data races not only concurrency bug
- Deadlocks
  - Circular waiting
  - Example

Other forms of errors

- Data races not only concurrency bug
- Deadlocks
  - Circular waiting
  - Future 1:
    - Holds lock1
    - Waiting for lock2
  - Future 2
    - Holds lock2
    - Waiting for lock1

```scala
def deadlocking(): Unit =
  val lock1 = new Lock()
  val lock2 = new Lock()
  Async.blocking:
    Future {
      lock1.lock()
      lock2.lock()
      lock2.unlock()
      lock1.unlock()
    }
    Future {
      lock2.lock()
      lock1.lock()
      lock1.unlock()
      lock2.unlock()
    }
```

McCCT deadlocks

- Why does it not always deadlock?
  - A single future may acquire both locks
  - Race for the lock
- Model locking as a schedule point
  - Acquire lock or let other future execute
- McCCT records:
  - Running Future
  - Status of Locks
  - => Recognize Deadlocks

```
def deadlocking(): Unit =
  val lock1 = new Lock()
  val lock2 = new Lock()
  Async.blocking:
    Future {
      lock1.lock()
      lock2.lock()
      lock2.unlock()
      lock1.unlock()
    }
    Future {
      lock2.lock()
      lock1.lock()
      lock1.unlock()
      lock2.unlock()
    }
```

McCCT deadlocks

- Why does it not always deadlock?
  - A single future may acquire both locks
  - Race for the lock
- Model locking as a schedule point
  - Acquire lock or let other future execute
- McCCT records:
  - Running future
  - Status of locks
  - => Recognize deadlocks

```
def deadlocking(): Unit =
  val lock1 = new Lock()
  val lock2 = new Lock()
  Async.blocking:
    Future {
      lock1.lock()
      lock2.lock()
      lock2.unlock()
      lock1.unlock()
    }
    Future {
      lock2.lock()
      lock1.lock()
      lock1.unlock()
      lock2.unlock()
    }
```

McCCT deadlocks

- Why does it not always deadlock?
  - A single future may acquire both locks
  - Race for the lock
- Model locking as a schedule point
  - Acquire lock or let other future execute
- McCCT records:
  - Running futures
  - Status of all futures & locks
  - => Recognize deadlocks

```
def deadlocking(): Unit =
  val lock1 = new Lock()
  val lock2 = new Lock()
  Async.blocking:
    Future {
      lock1.lock()
      lock2.lock()
      lock2.unlock()
      lock1.unlock()
    }
    Future {
      lock2.lock()
      lock1.lock()
      lock1.unlock()
      lock2.unlock()
    }
```

Under the hood

Interaction future ↔ CCT scheduler

Future creation

- Wrapper for `Future`
- `Controller` used to control execution:

```
val p = Promise[T]()
val controller = new Controller
val task = new Runnable:
  def run() =
    controller.await()
    val result = body(using controller)
    Scheduler.finish(controller)
    p.complete(Success(result))
Scheduler.startThread(task)
Scheduler.submit(controller)
```

Interaction future ↔ CCT scheduler

Future creation

- Wrapper for `Future`
- `Controller` used to control execution:
  - Beginning: wait for scheduler to enable execution

```
val p = Promise[T]()
val controller = new Controller
val task = new Runnable:
  def run() =
    controller.await()
    val result = body(using controller)
    Scheduler.finish(controller)
    p.complete(Success(result))
Scheduler.startThread(task)
Scheduler.submit(controller)
```

Interaction future ↔ CCT scheduler

Future creation

- Wrapper for `Future`
- `Controller` used to control execution:
  - Beginning: wait for scheduler to enable execution
  - Inform scheduler of termination

```
val p = Promise[T]()
val controller = new Controller
val task = new Runnable:
  def run() =
    controller.await()
    val result = body(using controller)
    Scheduler.finish(controller)
    p.complete(Success(result))
Scheduler.startThread(task)
Scheduler.submit(controller)
```

Interaction future ↔ CCT scheduler

Future creation

- Wrapper for `Future`
- `Controller` used to control execution:
  - Beginning: wait for scheduler to enable execution
  - Inform scheduler of termination
  - Submit new task to scheduler that is ready to run

```
val p = Promise[T]()
val controller = new Controller
val task = new Runnable:
  def run() =
    controller.await()
    val result = body(using controller)
    Scheduler.finish(controller)
    p.complete(Success(result))
Scheduler.startThread(task)
Scheduler.submit(controller)
```

CCT scheduler

- Maintains a **queue of ready-to-run tasks**
- Runs the main scheduler loop
  - Pseudo code on right-hand side
- Uses a **configurable exploration algorithm** `alg` to select the next tasks to run

```
while (true):
  lock.lock()

  while readyTasks.isEmpty
    && !hasFinished() do
      queueChange.awaitUninterruptibly()

  if hasFinished() then
    terminate.signal()

  executeTasks(getNextTasks(alg))

  lock.unlock()
```

Wrapping future await

- Code in blue = unwrapped code
- When current task is unblocked, submit controller to scheduler
  - Do not increment #tasks
- Wait for scheduler to resume the task

```
def await(using controller: Controller): T =
  val result = underlying.await()

  // resubmit the same controller
  // false = do not increment #tasks running
  Scheduler.submit(controller, false)

  // wait for scheduler to resume this task
  controller.await()

  result.get
```

Wrapping future await

- Code in blue = unwrapped code
- When current task is unblocked, submit controller to scheduler
  - Do not increment #tasks
- Wait for scheduler to resume the task

```
def await(using controller: Controller): T =
  val result = underlying.await()

  // uses given controller
  schedulePoint()

  result.get
```

Next steps

Next steps

- Automatic instrumentation
  - Automatic use of wrappers
  - Insertion of schedule points
  - Automatic use of McCCT locks
- Add support for more concurrency libraries

Conclusion

Conclusion

- CCT is a powerful concurrency testing approach
  - **Find bugs** by exploring interleavings
  - **Reproduce bugs** by replaying faulty schedules
- McCCT is a CCT tool for Scala
  - Support for testing futures and async/finish
  - In progress: support for testing actors
  - Under active development, no release yet
  - GitHub: https://github.com/phaller/mccct
- Send us your concurrency bugs! 😎

Thank you!

Q&A